Advanced PHP

Amjahdi Walid
2334452
ISI S5

TP 5 :

Exercice 1 — Constructeurs et héritage

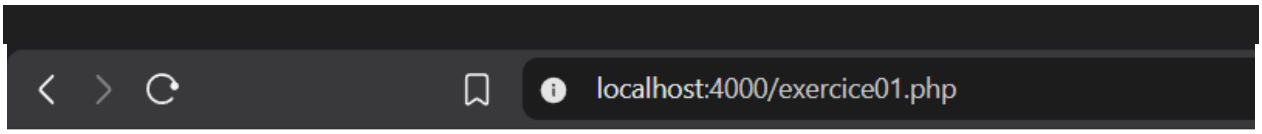
Le constructeur de SubClass appelle d'abord le constructeur parent, puis affiche son propre message. Résultat : In BaseClass constructor, puis In SubClass constructor.

Fichier : exercice01.php

```php
<?php
class BaseClass
{
    public function __construct()
    {
        print "In BaseClass constructor\n";
    }
}

class SubClass extends BaseClass
{
    public function __construct()
    {
        parent::__construct();
        print "In SubClass constructor\n";
    }
}

new SubClass();
```



In BaseClass constructor In SubClass constructor

Exercice 2 — Propriétés typées

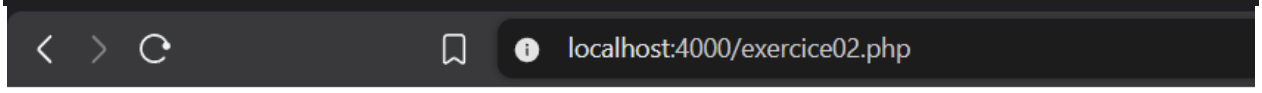
Oui, ce programme est correct à partir de PHP 7.4. Il initialise une propriété publique nom et une propriété privée age. nom est accessible depuis l'extérieur, tandis que age ne l'est pas directement. Le fichier affiche Alice.

Fichier : exercice02.php

```php
<?php
class Utilisateur
{
    public string $nom;
    private int $age;

    public function __construct(string $nom, int $age)
    {
        $this->nom = $nom;
        $this->age = $age;
    }
}

$user = new Utilisateur("Alice", 30);
echo $user->nom . PHP_EOL;
```



Alice

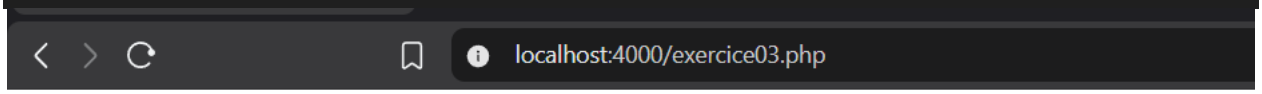
Exercice 3 — Promotion des propriétés

Oui, ce programme est correct à partir de PHP 8. Le constructeur déclare et initialise directement nom (public) et age (privé). C'est l'écriture courte de l'exercice 2. Le fichier affiche Alice.

Fichier : exercice03.php

```php
<?php
class Utilisateur
{
    public function __construct(public string $nom, private int $age)
    {
    }
}

$user = new Utilisateur("Alice", 30);
echo $user->nom . PHP_EOL;
```



Alice

Exercice 4 — readonly

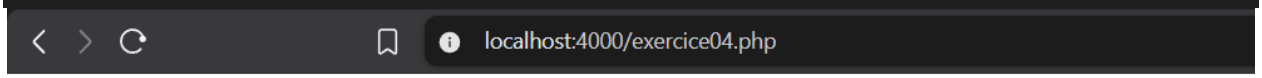
La classe et l'instanciation sont correctes à partir de PHP 8.1. echo $config->apiKey affiche 12345-XYZ. La tentative de modifier apiKey est incorrecte : une propriété readonly ne peut être écrite qu'à l'initialisation et déclenche une Error. Configuration::$apiKey est aussi incorrect, car la propriété n'est pas static.

Fichier : exercice04.php

```php
<?php
class Configuration
{
    public function __construct(
        public readonly string $apiKey,
        public readonly string $environnement
    ) {
    }
}
```

```php
$config = new Configuration("12345-XYZ", "production");
echo $config->apiKey . PHP_EOL;

// Incorrect after initialization: $config->apiKey = "nouvelle-clef";
// Incorrect: Configuration::$apiKey; (apiKey is not static)
```



12345-XYZ

Exercice 5 — __set et __get

La version complète est correcte. L'écriture de $obj->a déclenche __set(), puis sa lecture déclenche __get(). Résultat : Setting 'a' to '1', Getting 'a', puis 1. L'extrait du PDF semble omettre une accolade de fermeture ; le fichier fourni est la version corrigée.

Fichier : exercice05.php

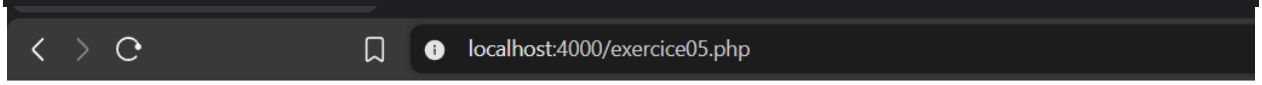
```php
<?php
class PropertyTest
{
    private array $data = [];
    public int $declared = 1;
    private int $hidden = 2;

    public function __set(string $name, mixed $value): void
    {
        echo "Setting '$name' to '$value'\n";
        $this->data[$name] = $value;
    }

    public function __get(string $name): mixed
    {
        echo "Getting '$name'\n";
        return $this->data[$name] ?? null;
    }
}

$obj = new PropertyTest();
$obj->a = 1;
```

```
echo $obj->a . PHP_EOL;
```



Setting 'a' to '1' Getting 'a' 1

Exercice 6 — self et static

Le programme est correct. self::quiSuisJe() est lié à ParentClass ; static::quiSuisJe() utilise la liaison statique tardive et appelle la redéfinition de EnfantClass. Résultat : Je suis ParentClass, puis Je suis EnfantClass.

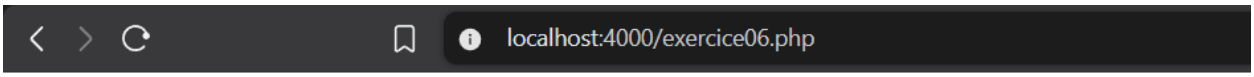
Fichier : exercice06.php

```php
<?php
class ParentClass
{
    public static function quiSuisJe(): void
    {
        echo "Je suis ParentClass\n";
    }

    public static function tester(): void
    {
        self::quiSuisJe();
        static::quiSuisJe();
    }
}

class EnfantClass extends ParentClass
{
    public static function quiSuisJe(): void
    {
        echo "Je suis EnfantClass\n";
    }
}

EnfantClass::tester();
```



Je suis ParentClass Je suis EnfantClass

Exercice 7 — Exemple __set et __get

Le programme est correct. L'assignation de prenom déclenche __set(), qui stocke Alice dans le tableau privé. Sa lecture déclenche __get(). Résultat : Assignation de 'Alice' à la propriété 'prenom', Lecture de la propriété 'prenom', puis Alice.

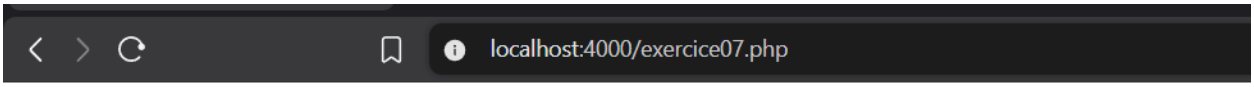
Fichier : exercice07.php

```php
<?php
class Utilisateur
{
    private array $donnees = [];

    public function __set(string $nom, mixed $valeur): void
    {
        echo "Assignation de '$valeur' à la propriété '$nom'\n";
        $this->donnees[$nom] = $valeur;
    }

    public function __get(string $nom): mixed
    {
        echo "Lecture de la propriété '$nom'\n";
        return $this->donnees[$nom] ?? null;
    }
}

$user = new Utilisateur();
$user->prenom = "Alice";
echo $user->prenom . PHP_EOL;
```



Assignation de 'Alice' à la propriété 'prenom' Lecture de la propriété 'prenom' Alice

Comparaison : version magique et classique

La version magique centralise des propriétés dynamiques dans un tableau privé et autorise la syntaxe directe $user->nom. Elle est flexible, mais moins explicite pour les IDE. La version classique déclare les propriétés et leurs getters/setters ; elle est plus explicite, mais plus verbeuse. Les deux fichiers affichent Nom : Alice et Email : alice@example.com.

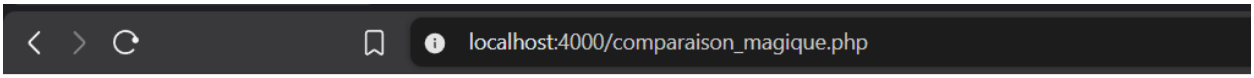
Fichier : comparaison_magique.php

```php
<?php
class UtilisateurMagique
{
    private array $data = [];

    public function __set(string $name, mixed $value): void
    {
        $this->data[$name] = $value;
    }

    public function __get(string $name): mixed
    {
        return $this->data[$name] ?? null;
    }
}

$user = new UtilisateurMagique();
$user->nom = "Alice";
$user->email = "alice@example.com";
echo "Nom : " . $user->nom . PHP_EOL;
echo "Email : " . $user->email . PHP_EOL;
```



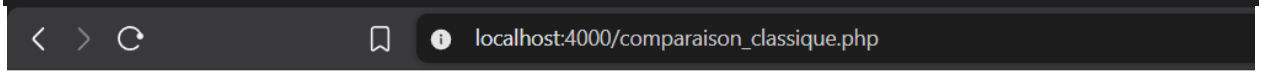
Nom : Alice Email : alice@example.com

Fichier : comparaison_classique.php

```php
<?php
class UtilisateurClassique
{
    private string $nom;
    private string $email;

    public function setNom(string $nom): void { $this->nom = $nom; }
    public function getNom(): string { return $this->nom; }
    public function setEmail(string $email): void { $this->email =
$email; }
    public function getEmail(): string { return $this->email; }
}

$user = new UtilisateurClassique();
$user->setNom("Alice");
$user->setEmail("alice@example.com");
echo "Nom : " . $user->getNom() . PHP_EOL;
echo "Email : " . $user->getEmail() . PHP_EOL;
```



Nom : Alice Email : alice@example.com

Exercice 8 — Stringable et __toString

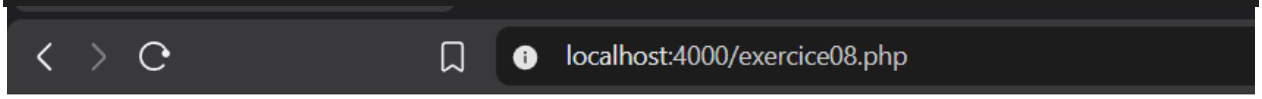
Le programme est correct à partir de PHP 8 une fois la mise en forme du PDF reconstruite comme class Utilisateur implements Stringable. __toString() est appelée automatiquement par echo. Résultat : Utilisateur: Alice.

Fichier : exercice08.php

```php
<?php
class Utilisateur implements Stringable
{
    public function __construct(private string $nom)
    {
    }

    public function __toString(): string
    {
        return "Utilisateur: " . $this->nom;
    }
}

$user = new Utilisateur("Alice");
echo $user . PHP_EOL;
```



Utilisateur: Alice

Exercice 9 — Countable

Le programme est correct. Countable permet d'utiliser count() sur l'objet ; la méthode count() retourne le nombre d'articles. Résultat : 3.

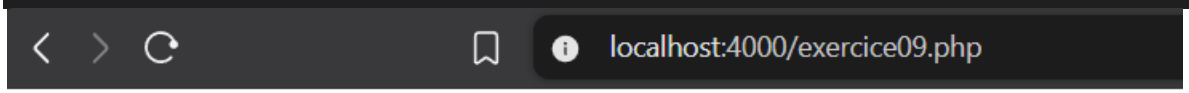
Fichier : exercice09.php

```php
<?php
class Panier implements Countable
{
    private array $articles = ['Pomme', 'Poire', 'Banane'];

    public function count(): int
    {
        return count($this->articles);
```

```
    }
}

$monPanier = new Panier();
echo count($monPanier) . PHP_EOL;
```



3

Exercice 10 — IteratorAggregate

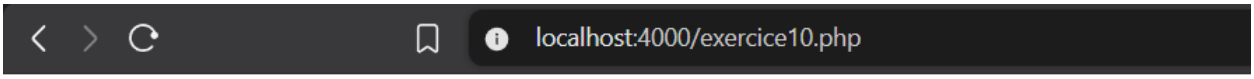
Le programme est correct. getIterator() retourne un ArrayIterator ; l'objet peut donc être parcouru avec foreach. Résultat : Léo Amine Sarah.

Fichier : exercice10.php

```
<?php
class Equipe implements IteratorAggregate
{
    private array $joueurs = ["Léo", "Amine", "Sarah"];

    public function getIterator(): Traversable
    {
        return new ArrayIterator($this->joueurs);
    }
}

$equipe = new Equipe();
foreach ($equipe as $joueur) {
    echo $joueur . " ";
}
echo PHP_EOL;
```



Léo Amine Sarah

Exercice 11 — Interfaces

Countable permet count($objet) et impose count(): int. Stringable permet la conversion d’un objet en texte et impose __toString(): string. IteratorAggregate permet foreach et impose getIterator(): Traversable, qui retourne un itérateur comme ArrayIterator.